

6CM504 Concurrency and communication

Modelling and verification

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

- 1 Introduction
- 2 The holy grail: concurrency and synchronisation
- 3 Parallel composition
- 4 Simple parallel: an example
- 5 Summary

This week

We have looked at the meaning of termination and deadlock—what it means for processes to successfully, or unsuccessfully terminate. However, until now, our processes have been characterised by a common theme: they were all sequential. This week we shall introduce concurrency and synchronisation—a theory of multiple things happening together.

Table of Contents

- 1 Introduction
- 2 The holy grail: concurrency and synchronisation**
- 3 Parallel composition
- 4 Simple parallel: an example
- 5 Summary

Systems of systems...

- ❶ Our systems have been rather uninteresting in the sense that they are single, sequential processes
- ❷ Realistic systems involve multiple, concurrent, co-operating systems
- ❸ In the following lectures, we will study how CSP handles concurrency:
 - Different models and types of concurrent interactions
 - How to compose concurrent systems
 - What sort of issues we may be interested in investigating in concurrent systems

What is concurrency?

Concurrency, (n): A running together in place or time; meeting, combination. Accordance in operation or opinion; co-operation; consent.
(Oxford English Dictionary)

What is synchronisation?

Synchronise (v): To occur at the same time; to coincide in point of time; to be contemporary or simultaneous. To combine or co-ordinate.

(Oxford English Dictionary)

Why does concurrency matter?

```
function V(sem S, int i):  
    [S <- S + 1]
```

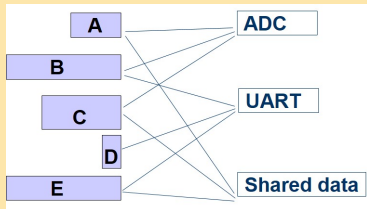
```
function P(sema S, int i)  
    repeat: [if S >= i :  
            S <- S - 1  
    break]
```



Remember this C code?

```
❶ int a;  
❷ void Task1() {  
❸     int x;  
❹     x= a;  
❺     movingAverage= MovingAverage(x);  
❻     a= movingAverage;  
❼ }
```

What about these shared resources?



Even our microcontroller environment is a whole melting pot of shared resources—the integration and management of these resources is down to the software we write.

Table of Contents

- 1 Introduction
- 2 The holy grail: concurrency and synchronisation
- 3 Parallel composition**
- 4 Simple parallel: an example
- 5 Summary

Alphabetised parallel

- 1 If P and Q are processes, and A some alphabet of events, then we write

$$P \parallel_A Q$$

to denote the parallel composition of P and Q

- 2 In a parallel composition processes evolve separately, while co-operating with others on the shared events in the set A

Interface parallel explained

- 1 Suppose we have the two processes P and Q, and the set of events A

$$P = a \rightarrow P'$$

$$Q = a \rightarrow Q'$$

$$A = \{a\}$$

- 2 We can create the process R that is the parallel composition

$$R = P \parallel_A Q$$

- 3 Now in observing R for the event a to happen, both P and Q must agree
- 4 In this example it is available as the first event in both P and Q so it may happen as the first observation of R

Stepping through interface parallel 1

- 1 Suppose we have the two processes

$$A = a \rightarrow A'$$
$$B = a \rightarrow B'$$

- 2 We can put them in parallel, looking like

$$(a \rightarrow A') \parallel \{a\} \parallel (a \rightarrow B')$$

- 3 This is equivalent to

$$a \rightarrow (A' \parallel \{a\} \parallel B')$$

- 4 In the trace of this process we will only observe one a happening—both processes have co-operated and engaged in it together
- 5 We refer to this as a synchronisation between the co-operating processes

Stepping through interface parallel 2

- 1 Suppose we have the two processes

$$B = b \rightarrow B' \quad \text{and} \quad C = c \rightarrow C'$$

- 2 We can put them in parallel, but without requiring them to synchronise on b or c

$$(b \rightarrow B') \parallel \{\} (c \rightarrow C')$$

which is equivalent to

$$\begin{aligned} & (b \rightarrow (B' \parallel \{\} (c \rightarrow C')))) \\ & [] \\ & c \rightarrow ((b \rightarrow B' \parallel \{\} C')) \end{aligned}$$

- 3 We can engage in either b or c as the first event. The processes do not synchronise on these events, so once one has occurred subsequent behaviour is either the initial event of the other process or the subsequent behaviour of the one whose initial event we engaged in

Stepping through interface parallel 3

- 1 Now assuming the same two processes

$$B = b \rightarrow B' \quad \text{and} \quad C = c \rightarrow C'$$

- 2 But this time, we synchronise on b

$$(b \rightarrow B') \parallel \{b\} \parallel (c \rightarrow C')$$

- 3 This is now equivalent to

$$c \rightarrow ((b \rightarrow B') \parallel \{b\} \parallel C')$$

- 4 The difference is that the event c must happen first because the process C is not in a state where it may engage in b , and yet both processes must agree on b for it to happen. The event b will only happen if at some time in the future C' is willing to perform it

Table of Contents

- 1 Introduction
- 2 The holy grail: concurrency and synchronisation
- 3 Parallel composition
- 4 Simple parallel: an example**
- 5 Summary

Loading boxes

- ① Two people are loading boxes onto a truck
 - There is a red box, a blue box, and a green box
 - Matthew is to load the red box while Marina is to load the blue box
 - The green box is particularly heavy and both of them will be needed to lift it
- ② What will this look like if we describe it as a system of CSP processes?

Loading boxes, pictorially



First, let's look at shared
(co-operating) events

Loading boxes, constraints

- 1 Matthew is very laid back—he is happy to load the red and green boxes in either order
- 2 Marina is less laid back—she insists that the green box is loaded before the blue
- 3 Question: can you write CSP processes that capture this behaviour, including these constraints above?

Loading boxes: Matthew

- 1 Matthew is very laid back—he is happy to load the red and green boxes in either order
- 2 Matthew might look like this

```
Matthew =  
  red -> green -> SKIP  
  []  
  green -> red -> SKIP
```

Loading boxes: Marina

- 1 Marina is less laid back—she insists that the green box is loaded before the blue

```
Marina =  
    green ->  
    blue -> SKIP
```

Loading boxes: all together now

- 1 Working together puts Matthew and Marina in parallel
- 2 Co-operating on the green box, which becomes our synchronisation

Removals = Matthew [|{green}|] Marina

Loading boxes: rewriting

- 1 We can rewrite our Removals process to help clarify why this works

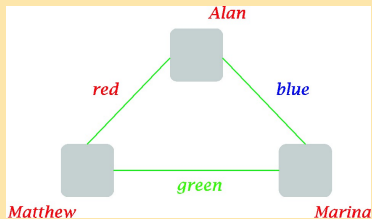
```
Removals =  
  red -> green -> blue -> SKIP  
  []  
  green -> ( red -> blue -> SKIP  
             []  
             blue -> red -> SKIP )
```


Loading boxes: more constraints

- ① We can introduce more constraints/behaviours simply by adding more processes in parallel
- ② Although the red box and the blue box are lighter than the green box, they are still heavy
 - Marina feels that she is doing her bit for Matthew by being involved with one box
 - So a third person is employed to assist with these boxes

```
Alan =  
  blue ->  
    red -> SKIP
```

Loading boxes, pictorially



Alan's help gives rise to a new set of shared events

Loading boxes: a challenge

- 1 Question: what do we get when we rewrite

```
(Matthew [|{green}|] Marina)  
[|{blue, red}|]  
Alan
```

as a single sequential process?

Table of Contents

- 1 Introduction
- 2 The holy grail: concurrency and synchronisation
- 3 Parallel composition
- 4 Simple parallel: an example
- 5 Summary**

Summary

- ➊ In this lecture we have been introduced to the concurrency operator
- ➋ This operator allows us to put processes in parallel, and specify which events are to be shared (synchronised) between the two processes
- ➌ Now we can start to build sophisticated models of communicating, interacting, co-operating processes!
- ➍ In the next lecture, we will see an example of a special case in concurrency—*interleaving*
- ➎ We will also look at a very famous case study, and investigate how FDR can help us find problems in a system that may go unobserved with testing alone

Summary

- ➊ Advance reading: Schneider, pages 42–48
- ➋ Advance reading: Roscoe, pages 65–76
- ➌ Advance reading: Woodcock and Davies, chapter 9